Objects Deep dive

```
class Student {
    ______ name - 4
    ______ age - 4
    ______ rollNo - 4
    ______ college - 4
}
```

= 16 bytes

Student S1 = new Student();

S1 → ref var stored in stack memory → $\boxed{4 \text{ bytes}}$ generally or 8 bytes

new Student() → stored in heap memory → more than 16 bytes

12 + 16 + 4 = 32 bytes

* Object size
  - Header size / object headers
  - exact fields
  - Padding

Object header → metadata

- Mark words
  - → lock
  - → sychronisation
  - → garbage collection
  - etc

  ⇓ $\boxed{8 \text{ bytes}}$

- Class pointers → 4 bytes or 8 bytes

total 12 bytes

Exact data/fields: here 16 bytes

Padding: 8 byte multiple

object header + Exact data = 12 + 16 = 28 bytes
+ 4 bytes of padding
= 32 bytes

```
class Person {          = object     + Exact data  + Padding
    byte age;             Metadata     (1 byte)      (3 byte)
}                         (12 byte)
```

= 16 bytes

* Call by value & Call by reference

1 Call by value

```
class Main {
    main() {                    [4]  [5]
        int x = 4;               x    y        call by value
        int y = 5;
        sout(x+" "+y)           // 4,5
        addTen(x, y)
        sout(x, y)              // 4,5
    }
    addTen(int x, int y) {      [4]  [5]       new Variable
        x += 10;                 x    y
        y += 10;
    }
}
```

differ^n

erased once func^n is done

so output doesn't changes

output doesn't changed

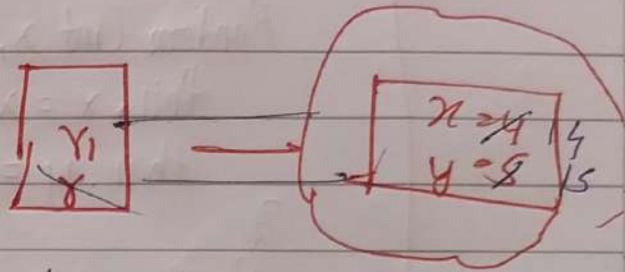
in reality java don't have call by reference

~~# class~~

// call by reference

```
class Demo {
  main() {
    Random r1 = new Random(4,5);
    sout (r1.x , r1.y)    // 4,5
    add Ten(r1)
    sout (r1.x, r1.y)     // 14, 15

  }
  addTen ( Random r) {
      r.x += 10;
      r.y += 10;
  }
}
class Random {
  int x;
  int y;
  Random(int x , int y){
     this.x = x;
     this.y = y;
  }
}
```

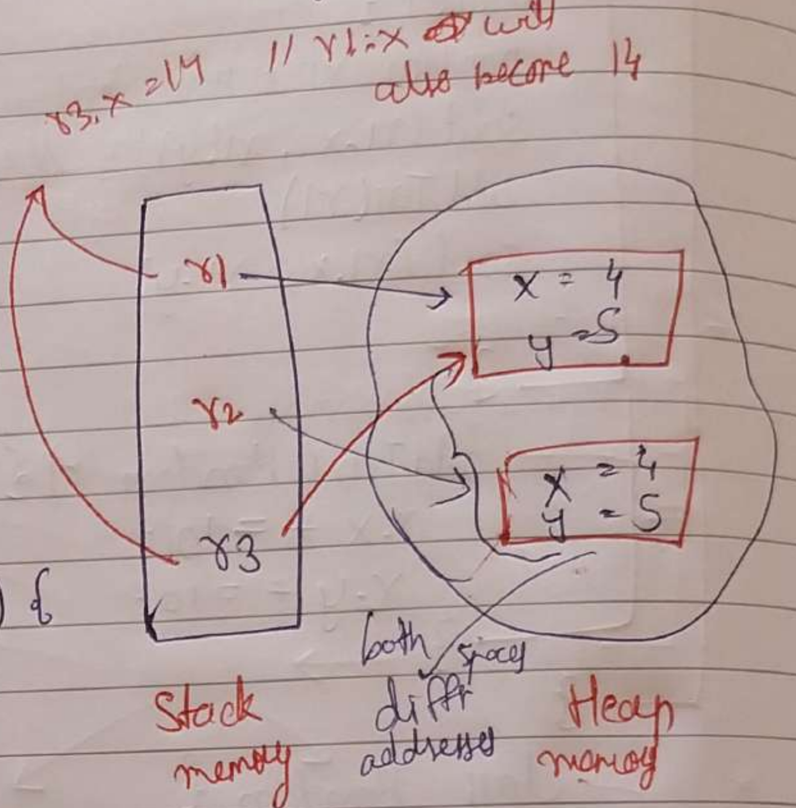


it is also call by value but since r1 & r both are reference variables ~~b~~ & both have same values so both point towards same address & hence value is changed

* Java ?
  - call by value ?
  - call by reference ?
  - Java is only call by value language.

*
```
class Random {
    int x;
    int y;

    Random (int x, int y) {
        this.x = x;
        this.y = y;
    }
    Random (Random r) {
        this.x = r.x;
        this.y = r.y;
    }
}
```




```
Random r1 = new Random (4,5);
Random r2 = new Random (r1); // copying of value
Random r3 = r1; // point same object
```

Deep Copy ← Random r2 = new Random (r1);

Shallow Copy ← Random r3 = r1;

both point same object means both will always have same value.
if change from r1 or r3, value will also change in r3 or r1